

Communication entre les services

Communication Synchrones — le service attend la réponse

Les 3 façons en Spring Boot :

RestTemplate — l'ancienne façon, encore très utilisée.

FeignClient — la façon élégante, déclarative.

WebClient — le moderne, non-bloquant.

Communication Asynchrone — le service n'attend pas

Message Broker (Kafka, RabbitMQ)

Event-Driven — variante de l'asynchrone

Communication Synchrones — le service attend la réponse

Le service A appelle le service B et **attend** la réponse avant de continuer.

REST (HTTP) — le plus courant. Un service expose des endpoints, l'autre les appelle via HTTP.

```
1 Service A → GET /users/123 → Service B
2           ← { id: 123, name: "Houda" }
```

✓ Simple, universel ✗ Si B est lent, A est bloqué. Si B tombe, A tombe.

gRPC — comme REST mais plus rapide. Utilise Protocol Buffers (binaire) au lieu de JSON. Très utilisé pour communication interne entre microservices. ✓ Très performant, typage fort ✗ Plus complexe à mettre en place

Les 3 façons en Spring Boot :

RestTemplate — l'ancienne façon, encore très utilisée.

```
1 RestTemplate restTemplate = new RestTemplate();
2 User user = restTemplate.getForObject(
3     "http://user-service/users/123",
4     User.class
5 );
```

✓ Simple, connu de tous ✗ Bloquant, verbeux, déprécié depuis Spring 5 ✗ Tu dois gérer l'URL manuellement

FeignClient — la façon élégante, déclarative.

```
1 @FeignClient(name = "user-service")
2 public interface UserClient {
3     @GetMapping("/users/{id}")
4     User getUserById(@PathVariable Long id);
5 }
```

Tu l'injectes et tu l'utilises comme un service normal :

```

1 @Autowired
2 private UserClient userClient;
3
4 User user = userClient.getUserById(123L);

```

✓ Propre et lisible — tu declares juste l'interface ✓ S'intègre avec Eureka pour la découverte de services ✓ Gestion automatique du load balancing ✗ Dépendance Spring Cloud

WebClient — le moderne, non-bloquant.

```

1 WebClient webClient = WebClient.create();
2 User user = webClient.get()
3     .uri("http://user-service/users/123")
4     .retrieve()
5     .bodyToMono(User.class)
6     .block(); // block() pour rester synchrone

```

✓ Non-bloquant par nature — meilleure performance ✓ Recommandé par Spring depuis Spring 5 ✗ Plus complexe si tu n'es pas familière avec la programmation réactive

i Pour la communication synchrone entre microservices en Spring Boot, on a plusieurs options. RestTemplate c'est l'ancienne approche, encore présente dans les projets legacy. FeignClient c'est plus élégant — tu declares juste une interface et Spring gère le reste, c'est ce qu'on utilise généralement avec Spring Cloud. Et WebClient c'est le moderne non-bloquant, recommandé pour les nouvelles applications.

Communication Asynchrone — le service n'attend pas

Le service A envoie un message et **continue sans attendre**. Le service B le traite quand il peut.

Message Broker (Kafka, RabbitMQ)

→ A publie un message dans un topic, B le consomme quand il est prêt.

```

1 Service A → [Message: "UserCreated"] → Kafka Topic
2                                     ← Service B consomme
3                                     ← Service C consomme aussi

```

✓ Découplé — si B tombe, le message attend ✓ Plusieurs services peuvent consommer le même message ✗ Plus complexe, pas de réponse immédiate

Event-Driven — variante de l'asynchrone

Les services communiquent via des **événements métier**. Au lieu de dire "fais ça", on dit "il s'est passé ça".

```

1 UserService → publie "UserRegistered"
2 EmailService → écoute "UserRegistered" → envoie email
3 BonusService → écoute "UserRegistered" → crée bonus

```

✓ Très découplé — UserService ne sait pas qui écoute ✓ Facilement extensible — tu ajoutes un service sans toucher les autres ✗ Difficile à déboguer, ordre des événements pas garanti